Artifact: A Reproducible Pipeline for Detecting Research Paradigms in the DELFI Community

Julian Dehne [1], Fabian Lochner [2] und Axel Wiepke [3]

Abstract: This artifact is a *dataset publication* accompanying the DELFI 2026 paper *Research Software Engineering in the Learning Technologies: a replication study of research paradigms in the DELFI community*. It releases the full set of intermediate datasets behind the study — the LLM annotation runs, the within- and cross-model aggregated labels, the human re-annotations, the reference-study evaluation, and the derived figures and tables — together with the pipeline that produces them. Every published number and figure is reproducible from the shipped datasets alone, *without* the copyrighted DELFI paper PDFs or a commercial LLM API; the steps that consume those sources are optional, because their outputs are themselves published as datasets. A programmatic guard enforces that no paper full text is ever contained in the released data. We describe the released datasets, the pipeline that regenerates them, the reproducibility guarantees, and the constraints that motivated the design for the artifact-evaluation track.

Resource Type: Data set (with an accompanying software pipeline)
License: Code: MIT; Data: CC-BY-4.0
Stable ID: DOI: 10.5281/zenodo.20681435
Repository: https://github.com/juliandehne/rse_in_delfi_artifact

Keywords: Research Software Engineering, Replication, Large Language Models, Inter-Coder Reliability, Reproducibility, DELFI

1 Introduction

The companion study [DLW26a] replicates and extends a 2017 analysis of research software paradigms across the DELFI proceedings, using three open-weight large language models (LLMs) to annotate $n = 798$ papers and validating the result against the earlier human-coded reference study. This artifact packages the *evidence and the computation* behind those results. The central design constraint is legal: the analyzed DELFI papers are copyrighted, so the artifact ships only derived data (labels, justifications, human re-annotations, bibliographic metadata) and is built so the paper- and API-dependent steps are optional while every published number and figure stays reproducible.

[1] Gesellschaft für Informatik, Politik und Wissenschaft, Weydingerstraße 14-16, 10178 Berlin, Deutschland, julian.dehne@gi.de, https://orcid.org/0000-0001-9265-9619
[2] Carl von Ossietzky Universität Oldenburg, Digital Social Science, Ammerländer Heerstraße 114-118, 26129 Oldenburg , Deutschland, fabian.lochner@uni-oldenburg.de, https://orcid.org/0009-0006-3157-5335
[3] University of Potsdam, An der Bahn 2, 14476 Potsdam, Germany, axel.wiepke@uni-potsdam.de, https://orcid.org/0000-0002-0555-4040

2 Artifact overview

The primary artifact is the dataset collection under `data/` (enumerated in the abstract). Accompanying it are a Python package (`pipeline/`: `config`, the six steps and a full-text guard) and the management script `run_pipeline.py`, which regenerate those datasets and figures from one another; every published number and figure is reproducible from the shipped datasets alone. We target the data *Available* and *Reusable* badges: the datasets are archived, openly licensed, documented with a data dictionary, and accompanied by the code that regenerates them.

3 The pipeline

The pipeline has six steps (Table 1); each step's output is a published dataset. Steps 3, 5 and 6 regenerate their datasets from the shipped data and run by default. Steps 1, 2 and 4 are *optional*: their outputs are already published, so they need only be re-run to rebuild those datasets from the original paper PDFs (steps 1 and 4) or the LLM API (step 2).

Tab. 1: Pipeline steps and default execution policy.

#	Step	Default	Requires
1	Preprocessing (PDF → text)	**optional**	paper PDF folder
2	LLM experiments	**optional**	SAIA/KISSKI API token
3	Descriptive analysis (figures)	runs	shipped data
4	Human re-annotation	**optional**	paper PDF folder (interactive)
5	Label aggregation (intra + inter, ICR)	runs	shipped data
6	Secondary analysis (paradigm biases)	runs	shipped data

Data flow. Step 1 extracts text from the PDFs; step 2 annotates each paper with three LLMs (three runs each) via the SAIA API. Step 5a aggregates the runs per model (majority vote) with *intra*-LLM reliability; step 4 elicits a human label where all three models disagree; step 5b aggregates across models (majority vote, ties broken by the human label), reports *inter*-LLM reliability and validates against the 2017 reference study. Steps 3 and 6 produce the figures and the bias analysis. `run_pipeline.py` runs the default steps; `--paper-folder` enables steps 1/4 and `--run-experiments` (with a SAIA token) enables step 2.

4 Reproducing the results

After `pip install -r requirements.txt`, `python run_pipeline.py` regenerates — from the shipped data, with no PDFs or API key — the final aggregated dataset (798×185), the ICR tables, the paradigm distribution and label trend figures, and the combined label bias tables. If the paper pdfs and the SAIA token are available, the full pipeline is reproducible.

5 Testing and AI Usage

The pipeline was tested with the minimal setup (3 paper per model) for stability. The data analysis was checked using whitebox testing and cross-comparison of the jupyter notebooks and the python scripts that were generated by Claude Code. AI was only used to generate build scripts and streamline the analysis scripts into the nicely parametrized python pipeline and for automatic annotation of the columns. The original paper and its accompanying data analysis was fully human-written. Future work on this framework could include metamorphic testing and unit tests for the computations of the reliability metrics.

6 Data and the no-full-text guarantee

The released data contain only labels, LLM/human justifications and bibliographic metadata. The original annotation script dropped the `abstract`, `text` and `references` columns before persisting results "for legal reasons"; the artifact preserves and enforces this invariant programmatically: `pipeline/fulltext_guard.py` scans every shipped table and fails if a banned full-text column is present or any cell exceeds a length threshold incompatible with a short justification. Step 1's full-text output goes only to a `.gitignore`'d directory.

The complete **codebook / data dictionary** is in `data/README.md`: it documents every column (name, type, value range, allowed values, missing-value coding), the column-name grammar, the data's provenance, and the (partially synthetic) LLM generation method for regenerating the labels.

7 LLM documentation

Per the DELFI AET LLM policy, Table 2 lists the three open-weight models used in step 2, all served through the KISSKI SAIA API. Each model was run three times (run_1–3). All runs used identical, deterministic inference parameters: **temperature** = 0, **top_p** = 1.0, **seed** = 42, no **stop sequences**, no explicit **max-tokens** cap (provider default), a 300 s request timeout, and no function/tool calling (plain JSON-mode prompting). The models are open-weight, so a closed-weight snapshot date does not apply; the snapshot is pinned by the version in the model identifier, and the run dates are recorded in the result filenames. The pipeline checks the SAIA API for the availability of the models (as they are quickly deprecated) and suggests the current version to be added as a parameter.

Prompts. The complete prompt — the system prompt and the user-prompt template with its `{row[...]}` placeholders — is shipped verbatim at `data/prompt_templates/prompt_template_1.md`. The placeholders `title`, `authors` and `year` resolve to public bibliographic metadata; the placeholders `abstract`, `text`

Tab. 2: LLM models and versions used for annotation (KISSKI SAIA API).

Short name	Model identifier (version)	Runs
mistral	`mistral-large-3-675b-instruct-2512`	3
llama	`llama-3.1-sauerkrautlm-70b-instruct`	3
gemma	`gemma-3-27b-it`	3

and `references` resolve to the copyrighted paper content and are therefore the *only* part of a fully resolved prompt that cannot be redistributed. The substitution logic (`pipeline/step2_experiments.py`, `fill_user_prompt`) is included so the resolved prompt can be regenerated by anyone holding the paper PDFs.

8 FAIR preparation and targeted badges

The artifact targets the **Available** and **Reusable** badges: *Findable/Available* via a stable archival DOI (Zenodo, from a tagged release) and machine-readable metadata (`CITATION.cff`); *Accessible* via open licensing (MIT code, CC-BY-4.0 data; `LICENSE`, `LICENSE-data`); *Interoperable* via open CSV with the documented codebook above; and *Reusable* via a parameterised, documented, re-runnable pipeline. No personal or human-subject data are released, so no further DSGVO/ethics statement beyond the no-full-text guarantee is required.

9 Requirements and environment

Python 3.11+, `pandas`, `numpy`, `scikit-learn`, `krippendorff`, `statsmodels`, `matplotlib` and `openpyxl`; see `requirements.txt`. `pymupdf` is needed only to re-run step 1, and an OpenAI-compatible client plus a SAIA token only to re-run step 2.

References

[DLW26a] Dehne, J.; Lochner, F.; Wiepke, A.: Research Software Engineering in the Learning Technologies: a replication study of research paradigms in the DELFI community. In: DELFI 2026 – Die 24. Fachtagung Bildungstechnologien der Gesellschaft für Informatik (GI). Lecture Notes in Informatics (LNI), Gesellschaft für Informatik, Bonn, 2026.

[DLW26b] Dehne, J.; Lochner, F.; Wiepke, A.: rse_in_delfi_artifact: A reproducible, database-free pipeline for detecting research-software paradigms in the DELFI community, https://github.com/juliandehne/rse_in_delfi_artifact, Replication artifact., 2026.